

Final Technical Report

Password Strength Checker

Python | Regex | Password Validation | Testing and Refactoring

Prepared By

Nicolle Sosa

Date

12/13/2026

Project No.

1

Repository / Project Link

github.com/nickisosa/password_strength_checker

REPORT PURPOSE

Use this report to summarize the completed project after development, refactoring, and testing are finished. Write in past tense when describing completed work. Keep the report focused on what was built, how it was tested, what was learned, and what could be improved.

1. Executive Summary

Summarize the completed project at a high level.

WRITING GUIDE

Write this section last. In 5-8 sentences, summarize the security need, what you built, how it works, how you tested it, and the final outcome.

Executive Summary

Keep this concise and professional. Do not list every coding step. Focus on the purpose, implementation, validation, and overall result.

The Password Strength Checker was developed as a Python cybersecurity training project to evaluate passwords based on a defined security policy. The program checks password length, uppercase and lowercase letters, numbers, and approved special characters before assigning a Weak, Medium, or Strong rating. It also provides suggestions when requirements are not met. The project used regular expressions, Boolean logic, scoring, manual testing, troubleshooting, and refactoring. A total of 24 test cases were completed and retested after refactoring to confirm that the program continued to produce the expected results. Overall, the project strengthened my skills in Python, password validation, testing, debugging, and technical documentation.

2. Problem, Security Need, and Objectives

Explain why the project was created and what it was expected to accomplish.

Problem / Security Need

Explain why weak passwords are a security concern and why password validation can support a basic security policy. Keep the scope realistic and avoid claiming the tool prevents every password attack.

Weak Passwords can create a security risk because passwords that are short or let character variety may be easier for attackers to guess or crack. Organizations often use password policies to establish basic requirements such as minimum length, uppercase and lowercase, letters, numbers, and special characters. The Security need for this project was to create a basic program that could evaluate whether a password meets a different set of password requirements. The password strength checker analyzes, the password, entered by the user and determines which requirements are met. It also provides a weak, medium, or strong rating and give suggestions when requirements are missing.

This project does not determine whether a password is completely secure against every type of attack. Instead, it demonstrates how basic password-security policies can be translated into program, logic and validated using python.

Project Objectives

Summarize the main objectives of the project, such as validating password length and character types, calculating a score, classifying strength, and providing improvement suggestions.

The main objective of this project was to create a python-based password strength checker that evaluate a user-entered password against a defined password policy.

The program was designed to check whether the password met the minimum length requirement and contained uppercase letters, lowercase, letters numbers, and approved special characters. The individual character checks were then combined into a score that was used to classify the password as weak, medium, or strong.

Another objective was to provide improvement suggestions when password requirements were not met. The project also focused on practicing regular expressions, Boolean logic, troubleshooting, manual, testing, refactoring, and technical documentation.

3. Technical Approach

Describe how the completed program works.

Program Design and Flow

Describe the program from input to output: password input, validation checks, Boolean results, character score, strength classification, suggestions, and final console output.

The program begins by asking the user to enter a password. It checks the password link and uses regular expressions to detect uppercase letters, lowercase, letters, numbers, and approve special characters. Each result is converted into a Boolean value and used to calculate a character score. The score and length requirement are then used to classify the password as Weak, Medium or Strong. If a requirement is missing, the program provides a suggestion for improvement.

Technologies and Concepts Used

Explain the role of Python, the re module, re.search(), regex character classes, Boolean values, len(), sum(), and if/elif/else in this project.

This project was built using Python and the re module for regular expressions. The program uses re.search() to detect uppercase letters, lowercase letters, numbers, and approved special characters. Boolean values are used to store validation results, while len() , sum() , and if/elif/else statements are used to check password requirements, calculate the character score, and assign the final strength rating.

4. Security Criteria and Policy

Document the exact rules enforced by the final version.

Password Validation Criteria

Document the minimum length and the four character categories checked by the program. Include the approved special-character policy.

The program checks five password requirements: a minimum length of 8 characters, at least one uppercase letter, one lowercase letter, one number, and one approved special character. Approved special characters include ! @ # \$ % & _ - * † The four character-based checks are used to calculate the character score.

Strength Classification Rules

State the final policy exactly as implemented: what qualifies as Strong, Medium, and Weak. Make sure this matches the final code.

A password is classified as strong when it meets the minimum length requirement and all 4 character checks. It is classified as medium when it meets the minimum length requirement and 3 out of 4 character checks. All other passwords are classified this weak.

5. Testing and Validation

Summarize how the final version was verified.

Testing Approach

Explain that the project was tested with planned test cases covering individual checks, scoring combinations, edge cases, and post-refactor regression testing.

The program was manually tested using 24 test cases that covered individual password checks, strength, scoring, special-character handling period, and edge cases. After refactoring, the test cases were repeated to confirm that the program still produced the expected results, and the code changes did not affect its behavior.

Test Results Summary

Summarize the overall results. Mention whether the documented test cases passed after refactoring and whether the program continued to produce the expected classifications and suggestions.

All 24 documented test cases were completed and retested after refactoring . The final version correctly handled, strong, medium, and weak classifications, individual character checks, special-character validation, and edge cases. The retesting confirmed that the reflector code continued to produce the expected results.

6. Troubleshooting and Refactoring

Document important problems solved during development.

Key Issues and Resolutions

Describe the most important troubleshooting moments, such as checking the wrong variable for special characters or attempting to sum `re.Match` objects instead of Boolean values. Explain the fix and how it was verified.

One issue occurred when the special-character check was testing the wrong variable, which caused incorrect results. This was fixed by checking the result of `re.search()` instead.

Another issue occurred when regex match objects were used in the scoring calculation, causing a `TypeError`. The search results were converted to Boolean values before being added to the score. The regex pattern for special characters was also corrected so symbols such as the hyphen and asterisk were handled properly.

Refactoring Summary

Explain how the final code was cleaned up: removing redundant passed/failed variables, converting regex results to Boolean values, improving variable names, and reusing stored Boolean results in the strength logic.

The code was cleaned up by removing unnecessary passed and failed variables and using Boolean values directly for each validation check. Variable names were also improved to make the code easier to read. The scoring logic was simplified so the stored Boolean results could be used directly in `character_score` without changing the programs behavior.

7. Results, Security Considerations, and Limitations

State what the finished tool accomplished and where its scope ends.

Final Results

Summarize what the completed program successfully does and what the final testing demonstrated.

The completed password strength checker successfully evaluates passwords, using the defined length and character requirements. It correctly assigns weak, medium or strong ratings and provide suggestions when requirements are missing. Final testing confirmed that the program worked as expected after factoring and handled the documented cases without errors.

Security Considerations

Discuss safe use of synthetic test passwords, avoiding storage of real credentials, and the fact that this tool evaluates composition rather than overall account security.

This project only evaluates password composition and does not guarantee that a password is fully secure. Real passwords should not be used during testing or stored and screenshots, source code, or documentation. The checker does not test for breached passwords, password reuse, common passwords, or resistance to every type of password attack.

Limitations

Explain what the checker does not evaluate, such as breached-password history, common-password lists, password reuse, entropy, dictionary words, or every possible attack method.

The program uses a basic password policy and only checks length and character composition. It does not measure password entropy, check for common or breached, passwords, detect, password, reuse, or compare passwords against dictionary lists. The application is intended as a learning project and would need additional security features before being used in a real-world environment.

8. Lessons Learned and Future Improvements

Reflect on what you learned and what you would improve next.

Lessons Learned

Write about the specific skills you practiced: Regex, `re.search()`, Boolean logic, validation, scoring, debugging, refactoring, test cases, and technical documentation. Focus on what became clearer through hands-on work.

This project help me better understand how password policies can be turned into program logic. I gained hands on a practice with regular expressions, `re.search()`, Boolean values, conditional statements, scoring logic, debugging, and testing. I also learned the importance of refactoring code without changing its behavior and documenting test results clearly. One of my biggest areas of growth was becoming more comfortable, reading errors, identifying what caused them, and correcting the logic step-by-step.

Future Improvements

List realistic next steps for a later version, such as functions, automated unit tests, stronger policy options, breach checks, hidden password input, or an object-oriented version.

Future improvements could include organizing the program into reusable functions, adding automated unit testing, and hiding password input while the user types. The project could also be expanded to check for common or breached passwords, support, stronger or customizable, password policies, and eventually be rebuilt, using object oriented programming, or a graphical user interface.

9. Conclusion and Recommendation

Close the report with the final project outcome.

Conclusion

Summarize what the project demonstrated about translating a security policy into code, validating input, testing behavior, and improving code quality through refactoring.

This project showed how basic security policy can be translated into working Python code. The "Password Strength Checker," successfully validate password requirements, calculate a character score, assigned a strength rating, and provides improvement suggestions. Through testing, troubleshooting, and refactoring, I was able to improve both the program and my understanding of secure coding practices. The project also helped strengthen my skills in Python, regular expressions, testing, and technical documentation.

Recommendation / Next Step

If this were reviewed by a supervisor, state what should happen next: additional testing, code review, stronger policy requirements, automated testing, or future integration.

The next step for this project would be to improve the program by organizing the code into functions and adding automated unit tests. Future versions could also include stronger password checks, hidden password input, or breached-password detection. Before any real-world use, the application should go through additional security review and testing.